VM Backend

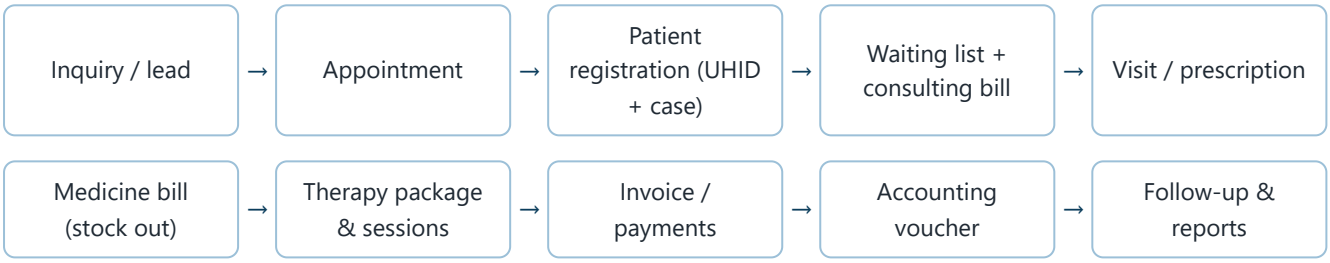
Project feature flows, one feature at a time

Laravel API (core app + `Modules/Patient`)
Branch: `dev` · Prepared 31 Oct 2026
Source: route files, controllers and services in the repository

Contents

1. System overview & request pipeline
2. Clinic signup & onboarding
3. Login, users & roles
4. Branches, subscription & plan gating
5. Settings & master data
6. Inquiry (lead CRM) pipeline
7. Appointments & calendar
8. Patient registration
9. Waiting list & consulting charges
10. Visit / prescription
11. OPD periods (OPD vs IPD)
12. Therapy (Panchakarma) packages
13. Diet charts
14. Patient certificates
15. Billing & invoices
16. Medicine bill (pharmacy sale)
17. Inventory & purchasing
18. ERP accounting
19. Printing & messaging
20. ABDM / ABHA integration
21. Dashboards & reports
22. Activity log & mobile app updates

End-to-end patient journey



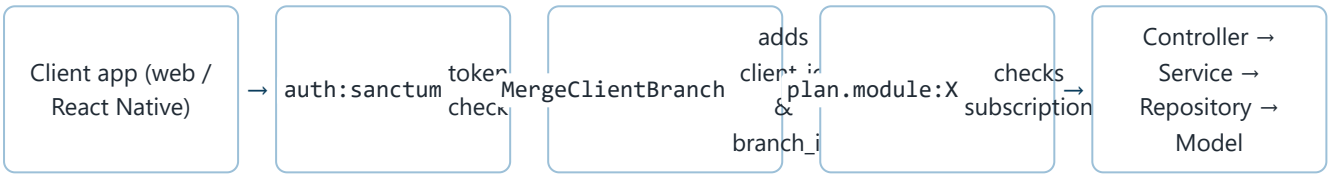
1. System overview & request pipeline

The backend is a Laravel REST API for Ayurveda, Homeopathy and other clinics. Shared business features (auth, billing, inventory, accounting) live in `app/`. Clinical features live in the `Modules/Patient` module.

Multi-tenancy model

Level	What it is	How it's identified
Client	The clinic / organisation that signed up	`client_id` (from logged-in user)
Branch	A physical centre of the client; most data is branch-scoped	`X-Branch-ID` header, else `user.current_branch_id`
Software type	Ayurvedic OPD, Homeopathy OPD, IPD Hospital, Multi-branch OPD/IPD, College…	`branches.software_type_id`

Every authenticated request

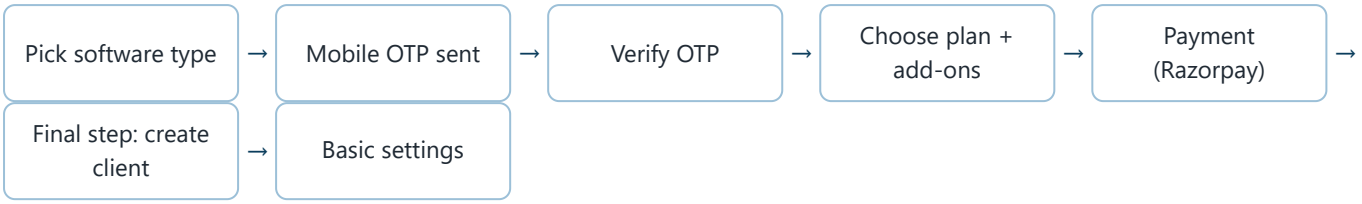


Responses use `ApiResponse::success / error` and API Resources. Most writes run inside `DB::transaction`, and many are wrapped in grouped activity logging.

Plan-gated modules

Routes tagged `plan.module:<KEY>` (and some services through `PlanGate::assertModule`) return a "module locked" error when the client's active subscription doesn't include the feature. Keys used: `BILLING, INVENTORY, ACCOUNTING, THERAPY, DIET, ANALYTICS, BRANCHES, ABHA`. `PlanGate::consume` also enforces monthly quotas (users are counted overall).

2. Clinic signup & onboarding



1. `GET /software-type` lists clinic types. `GET /pricing-plans`, `/pricing-plans/{id}` and `POST /addon/plans` return plans and add-ons.
2. `POST /signup/Otp` validates the Indian mobile number, rejects numbers that are already registered (409), stores a 6-digit OTP in `signup_otp_auths` and sends it by SMS. It returns an `auth_id`.
3. `POST /signup/verifyOtp` compares the OTP and sets `is_verified = 1`.
4. `POST /payment/create-order` → `POST /payment/verify` handle payment for paid plans.
5. `POST /signup/final` (or `/sign-up`) runs in one transaction. It creates the **Client** (with logo), the first **Branch** (with software type), the branch **address**, the admin **User**, a default "OPD" doctor department, roles, the subscription with its features, and seeded master data.
6. `POST /signup/basic-setting` (after login) saves consultation fees, follow-up days and morning/evening time slots for OPD, therapy, IPD and OT.

`/direct-signup` (`DirectRegistrationService`) is a shortcut that provisions everything in one call.

3. Login, users & roles

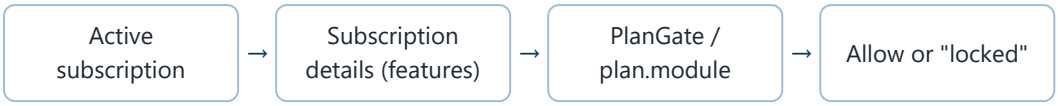
Authentication

1. `POST /login` or `POST /login/google` returns a Sanctum token. The login is recorded in the login log.
2. `GET /auth/me` and `GET /user` return the current user, branch and permissions.
3. Forgot password: `POST /reset-password-otp` → `POST /reset-password`. Logged-in users use `POST /change-password`. `POST /logout` revokes the token.

Users & roles

1. `/setting/role`: create roles and assign module permissions (`/role/modules` lists permission keys from `config/module_permissions.php`). System roles are read-only.
2. `/setting/user`: CRUD for staff (doctors, receptionists, therapists…). Users are linked to doctor departments and branches. The user count is checked against the plan quota.
3. `/doctor/profile/{id}` and `/doctor/save-profile/{id}`: the doctor's profile for prints (signature, qualifications).

4. Branches, subscription & plan gating



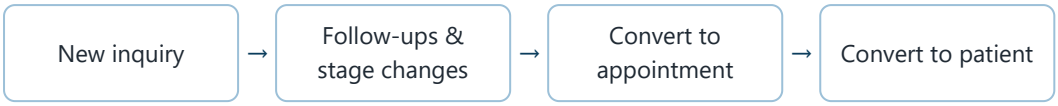
1. `/branch` : list, view, update and delete branches. Creating a branch requires the **BRANCHES** module.
2. `GET /my-subscription` shows the current plan. `POST /subscription/upgrade` upgrades it and adds add-ons. Branch subscriptions are synced by `BranchSubscriptionSyncService` .
3. `GET /wallet` : the client's wallet balance and transactions (used for SMS/WhatsApp credits, among other things).
4. `/communication` : log of messages sent to patients.

5. Settings & master data

Branch admins configure these before daily use. Most are simple CRUD endpoints (`GET` list, `POST` create, `POST /{id}` update, `DELETE /{id}`).

Area	Endpoints (prefix)	Purpose
Clinical masters	`setting/symptoms, habits, habits-options, diagnosis, modern-diagnosis, modern-system, present-illness`	Pick-lists used in the prescription
Departments	`setting/consultation-department`	Doctor departments and sub-departments
History forms	`setting/history-form-master`	Dynamic question groups (TEXT, RADIO, CHECKBOX, *_WITH_DETAIL) shown in visits
Registration form	`setting/patient/registration-form`	Turns vitals, illness history, additional info and ID proof on or off
Therapy masters	`setting/therapy, therapy-category, therapy-room, therapy-package`	Therapies, rooms and ready-made packages (THERAPY)
Diet masters	`setting/diet/*`	Categories, items, translations, languages, do's/don'ts, plan templates (DIET)
Certificates	`setting/certificate-type, certificate-variable`	Certificate templates, fields and approval rules
Pharmacy masters	`setting/medicine, medicine-type, unit, company, composition, inventory-category`	Medicine catalogue, units, packs and conversions (INVENTORY)
Other	`setting/source-panel, lead-source, lab-report, special-case, expense-category`	Referral panels, lead sources, lab tests, expense heads
Numbering	`setting/unique-number`	Prefix, year, department and zero padding for UHID, case, invoice, IPD and other numbers
Messaging	`setting/message/whatsapp, sms`	Provider credentials and which alerts are switched on
Calendar / print	`setting/appointment-calendar , /print-settings`	Slot settings, print header/footer, page layout

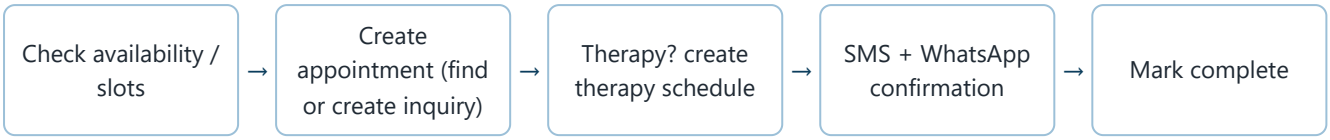
6. Inquiry (lead CRM) pipeline



1. `POST /inquiries` captures name, phone, source mode, lead source, inquiry type (treatment), references and assigned user/doctor/department.
2. Stages (`Inquiry::STAGE_*`): New → In progress → Interested / Not responding / Not interested → Appointment booked → Visited → Converted, or Dead.
3. `POST /inquiries/{id}/stage` and `/priority` update the lead. `/followups` stores call notes and the next follow-up date (buckets: today, upcoming, overdue).
4. `GET /inquiries/pipeline` returns the count per stage. `/report` is the analytics view (ANALYTICS).
5. `POST /{id}/convert-appointment` and `/{id}/convert-patient` hand off to the next flows.

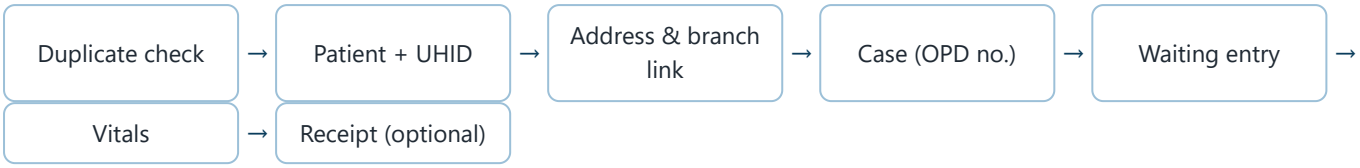
> An inquiry is the single contact record. Appointments, walk-in medicine bills and invoices all call `Inquiry::findorAdd` , and patient save keeps the inquiry name and phone in sync (1 patient = 1 inquiry). When a patient registers from an inquiry, the inquiry's stage changes to **Converted**.

7. Appointments & calendar



1. `GET /appointments/availability` and `/slot-availability` calculate free slots from the calendar settings, doctors, rooms and existing bookings (`AppointmentSlotAvailabilityService`).
2. `POST /appointments` (`AppointmentService::create`):
   - Links an inquiry, or creates one from the patient name and mobile.
   - **Consultation** (type ≠ 1) creates one appointment per selected doctor.
   - **Therapy** (type = 1) creates one appointment and a `PatientScheduleTherapy` record (therapies, therapists, room, medicines).
   - Rejects slots that clash (`validateAvailabilityOrFail`).
   - Sends SMS + WhatsApp confirmation for future appointments only.
3. `POST /appointments/bulk` creates many in one go. Calendar views: `/by-date` , `/by-date-range` , `/by-month` , `/datatable` , `/today-stats` .
4. `POST /appointments/{id}/complete` marks it done. Editing an appointment keeps its ID (`replaceKeepingId`).

8. Patient registration



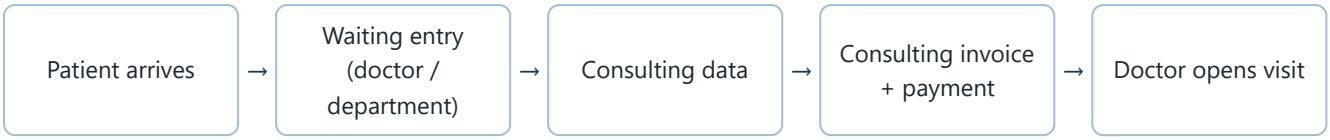
`POST /patient` (`PatientController::store`) runs in a single transaction:

1. Rejects a duplicate first name + last name + mobile (or WhatsApp) number with a 422.
2. Reads the branch's **registration settings** to decide whether vitals, illness history, additional info and ID proof are captured.
3. `PatientService::create` generates the **UHID** (`UniqueIdGeneratorService`), stores the photo and ID proof uploads, and syncs or creates the linked **inquiry**.
4. Creates the address, the patient↔branch link, and the **PatientCase** (the case / OPD number). Sets `patient.current_case_id`.
5. Adds the patient to today's **waiting list** and saves **vitals** if they're turned on.
6. If an amount was entered, creates a combined **receipt** with consultation, registration and other-charge lines, discount and outstanding amount.

Related endpoints

- `POST /patient/{id}/case` opens a new case for an existing patient (copies the department from the last case unless one is given).
- `GET /patient/{id}/cases-overview` returns cases, diagnosis, expected treatment days and the latest Ayush insurance snapshot.
- `GET /patient/search`, `/datatable`, `/{id}/account-statement` (BILLING), `POST /{id}/sync-credit`.

9. Waiting list & consulting charges



1. `POST /patient/waiting/save` creates the waiting entry. If no doctor or department is given, it copies them from the case's last waiting entry (or uses the logged-in user). College software types also get daily, monthly and yearly serial numbers.
2. `POST /patient/waiting/save-with-consulting` does three things in one transaction: creates the waiting entry, creates `PatientConsultingData` (charges, discount, payment mode, bank details), and creates a **consulting invoice** with its receipt.
3. `POST /patient/waiting/list` is the doctor's queue. `POST /waiting/{id}` updates status.
4. Deleting a waiting entry also deletes its consulting rows and invoices, then recalculates the patient's case credit.

10. Visit / prescription

The visit is the doctor's main screen. `POST /patient/visit` (create) and `POST /patient/visit/{id}` (update) go through `PatientVisitService` inside a transaction, then `handleVisitRelations` saves every part of the prescription:

Part	What happens
Diagnosis	Comma-separated Ayurveda, modern and modern-system names are upserted into the masters and linked in `patient_diagnoses` (type 1/2/3)
Symptoms	Upserted from the payload. `deletedSymptomIds` removed on update
Medicines	Prescribed medicines (dose, anupan, duration). `deletedMedicineIds` removed on update
Therapy advice	If `advice_therapy = 1` (THERAPY required): creates or updates the advised therapy package, building a custom master package if a name is given. Turning the flag off deletes it
Documents	Prescription files uploaded with title and category. Metadata editable
Insurance	Ayush policy fields. Moving the status to 1 sends insurance messages
History form	`history_form_answers` and `followup_assessment_answers` synced to `patient_visit_history_answers`
Diet chart	`diet_chart` payload upserted (DIET required). See section 13
Lab advice	`advice_lab_report_ids` synced to the pivot table
Follow-up	Next follow-up date and remarks synced to the follow-up remark list

Reading visits

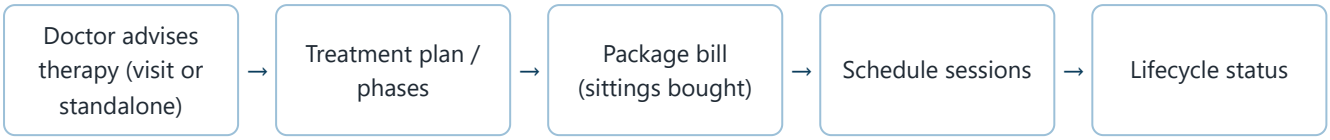
- `GET /patient/visit/list/{case_id}` lists the case history. `GET /patient/visit/{id}` returns `PatientVisitResource`, with nested history answers, the diet chart, lab IDs and consulting data.
- `POST /visit/last-prescription` and `/last-medicine-prescription` repeat the previous prescription.
- `GET /visit/followup-list` lists follow-ups due. `GET /patient/recent-list` lists today's or a date range's visits.

11. OPD periods (OPD vs IPD)

1. **OPD** = out-patient consultation. Every `PatientCase` starts as OPD (`is_ipd = 0`).
2. `OpdPeriodService` opens the first OPD period when a case is created. On each new visit it reuses the open period if the visit is on the same calendar day. Otherwise it closes that period and opens a new generated OPD number.
3. `resolve()` and `resolveForVisit()` map any activity time to its OPD number (start ≤ time < end).
4. **IPD** = in-patient admission. Right now this is only the `patient_cases.is_ipd` flag, `current_ipd_id`, an IPD number series and IPD time-slot settings. Reports split OPD and IPD by `is_ipd`.

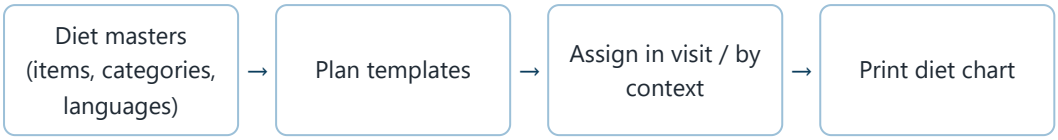
> There's no admission, bed or ward module or IPD billing source yet. The Day End report shows IPD revenue as 0 and says so.

12. Therapy (Panchakarma) packages



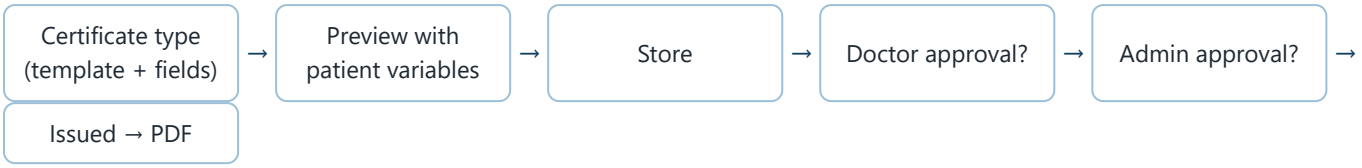
1. **Advice:** created from the visit (section 10) or with `POST /patient/therapy/advice`. Each *slot / phase* becomes a `PatientTherapyGroup` with `advised_sittings`, therapies (`PatientTherapyGroupItem`) and voice notes.
2. **Treatment plan:** `/therapy/{id}/advice-treatment-plan`, `/treatment-plan` and `/packages-by-case`. Phases can be adjusted (`/group/{groupId}/advised-days`) or deleted.
3. **Billing:** `POST /patient/therapy` in bill mode stores purchased sittings, rate, discount and GST per slot. It creates an invoice whose items point to each therapy group. Phase billing status compares prescribed vs purchased sittings and what's still due. Therapy medicines are deducted from stock (`applyStockOutForTherapyInvoiceLine`).
4. **Scheduling:** `/patient/schedule-therapies` books sessions (date, time, therapists, room). They're also created from therapy appointments and kept in sync by `PatientTherapyGroupAppointmentSyncService`.
5. **Lifecycle:** `POST /schedule-therapies/items/{id}/status` and `/medicines/{id}/status` mark therapy items and medicines as done. `/lifecycle-report` shows progress, split OPD/IPD.
6. `GET /therapy/{id}/pdf` prints the package. `/advised-therapies/grouped` lists advice not yet converted into a bill.

13. Diet charts



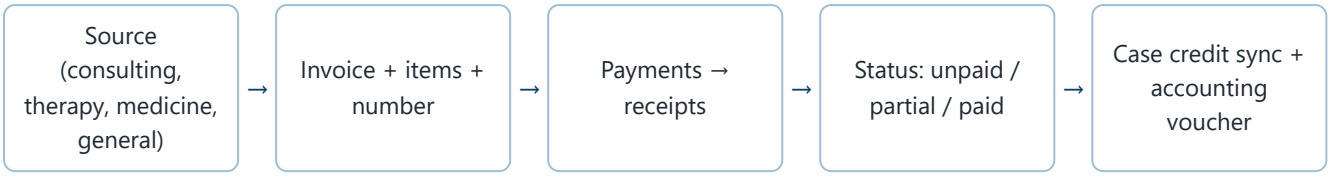
1. Masters under `setting/diet/*`: categories, items, item translations (multi-language), do's and don'ts, plan categories, plan items, category instructions.
2. In the visit, `GET /visit/diet/master-data` and `/search-diseases` load the options. The chart is saved with the visit's `diet_chart` payload, or by context with `/visit/diet/by-context` and `/diet/clear`.
3. Data: `PatientPrescriptionDiet` (one per visit, the `PatientVisit::dietChart` relation) with `items`. Each item's status is Take (1), Avoid (2) or Occasional (3).
4. Standalone assignment: `/patient/diet/assign` (list, show, store, delete).
5. Printing: `/patient/prescription/diet-chart/print` and `/diet/assign/print` return preview and PDF links (`/print/diet_chart/{id}`).

14. Patient certificates



1. **Setup:** `setting/certificate-type` defines the body template, footer, custom fields, whether the body can be edited at issue time, and whether doctor and/or admin approval is required. Variables such as patient name, diagnosis and IPD number are resolved by `CertificateVariableResolverService`.
2. **Issue:** `GET /patient/certificate/types/{id}/form` → `POST /certificate/preview` → `POST /certificate`. Storing validates required fields, generates the certificate number, renders the template, sets the starting status, and writes an audit entry.
3. **Statuses:** draft → pending_doctor_approval → pending_admin_approval → issued, or cancelled. Only a doctor can approve the doctor stage and only an admin can approve the admin stage (`POST /{id}/approve`). `POST /{id}/cancel` requires a reason.
4. `GET /patient/certificate/{id}/pdf` returns the printable certificate.

15. Billing & invoices



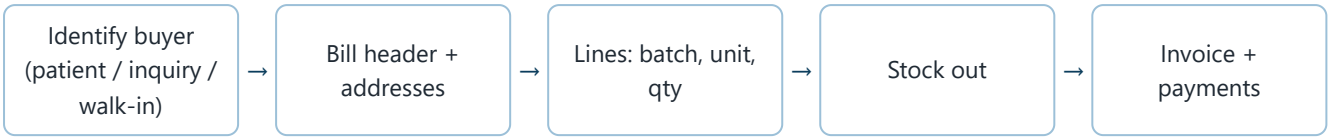
`InvoiceService::create` runs in a transaction with grouped activity logging:

1. **Estimate** (`is_estimate`): gets an ESTIMATE number, stays unpaid and records no payments.
2. **Normal invoice**: gets an INVOICE number per bill category. Balance = net − paid. Status is 1 (paid) when balance ≤ 0, 2 (partial) when 0 < balance < net, otherwise 0 (unpaid). Credit settlements from the patient's advance can pay part of it.
3. **Branch transfer** invoices carry no payment and are marked paid.
4. Saves items. Each payment becomes a `receipt_data` row (mode, reference number). Then syncs the inquiry, the patient case credit and the consulting data.
5. Calls `AccountPostingService::postInvoicePayment` . A missing ledger mapping is logged and doesn't block the invoice.

Other invoice actions (`/invoice` , BILLING)

Action	Endpoint	Effect
Pay	`POST /invoice/{id}/pay`	Adds a receipt, updates paid, balance and status, reposts the voucher
Collect dues	`POST /invoice/collect`	One payment spread across several outstanding invoices, using unlinked advance receipts first
Refund	`POST /invoice/refund`	Refund receipt (type 6) plus its own refund voucher
Cancel / delete	`POST /invoice/cancel/{id}` , `DELETE /invoice/{id}`	Reverses receipts, stock and vouchers
Reports	`/invoice/report` , `/medicine-bill-report` , `/patient-revenue-dues` , `/gst-report` , `/receipt/report`	Sales, dues, GST (by medicine / HSN / GSTR) and receipts

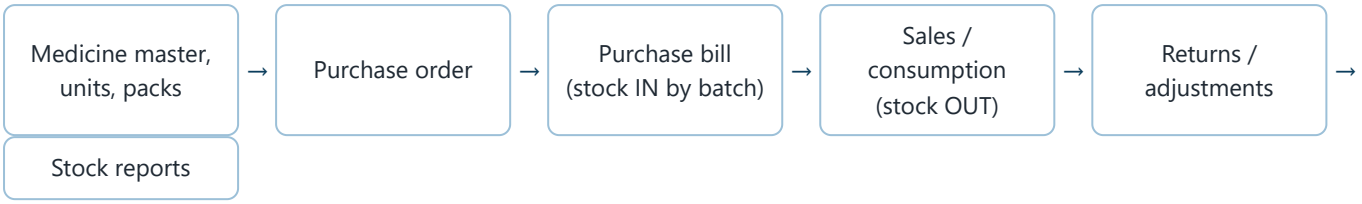
16. Medicine bill (pharmacy sale)



`POST /patient/medicine-bill → PatientMedicineBillService::create`:

1. Generates a medicine invoice number (not for estimates).
2. Resolves the buyer: an existing `patient_id`, a chosen `inquiry_id`, or a free-text walk-in. A walk-in creates an inquiry flagged `from_billing`. A **branch transfer** needs no buyer and deducts stock from the source branch.
3. Creates the bill header (amounts, discounts, tax, courier charges) with billing and courier addresses.
4. For each slot: normalises the sale unit, splits composition medicines into their ingredients, resolves batch, pack and base quantity, and normalises expiry. Stock is reduced through the inventory stock service. Negative stock is allowed only when `allow_negative_stock` is set.
5. Builds the invoice and its items through `InvoiceService`, which also handles receipts, credit and accounting.

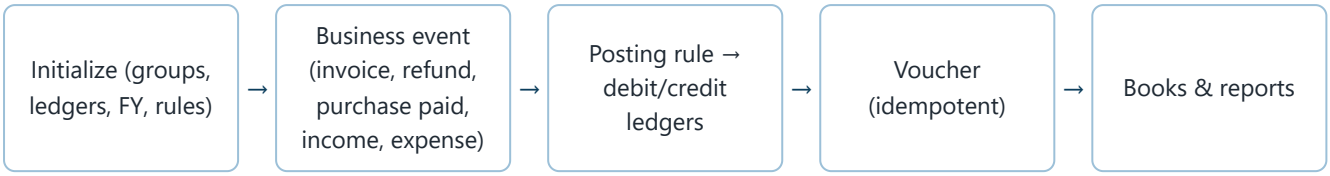
17. Inventory & purchasing (INVENTORY)



1. **Masters:** medicines with type, company, composition, HSN/GST, unit mappings and conversions (for example strip ↔ tablet), and packs. `POST /inventory/convert-qty` converts between units.
2. **Suppliers:** `/suppliers`.
3. **Purchase order:** `/purchase-order`. `GET /{id}/prefill` → `POST /{id}/create-bill` turns a PO into a bill and updates the received quantities and PO status.
4. **Purchase bill:** `POST /purchases` (`PurchaseService::create`) generates the bill number, links the PO, and creates items with **batches and stock movements** (`PurchaseItemService`). Updating a bill reverses the old movements first. `POST /purchases/{id}/pay` records payment, which posts an accounting voucher. `/bulk-add` adds many items at once.
5. **Purchase return:** `/purchase-return/item-details` → `POST /item-action` returns items and reduces stock.
6. **Internal consumption:** `/inventory/consumptions` (clinic use).
7. **Adjustment & import:** `POST /medicine-stock/adjust`. `/inventory/import` previews and imports opening stock from a spreadsheet.
8. **Reports:** medicine stock, batches, low stock, stock values, stock history, stock ledger, stock statement, purchase & returns report.

All stock changes go through `InventoryStockService::applyStockMovement / reverseStockMovement / applyStockDelta`, so each change is a ledger row that can be reversed.

18. ERP accounting (ACCOUNTING)



1. `POST /accounting/initialize` seeds the chart of accounts, voucher types, financial year and default posting rules. `/backfill` creates vouchers for existing data.
2. **Posting rules** map *(source_type, posting_event, category)* to debit and credit ledgers. Payment modes map to the Cash, Bank, UPI or Card ledger.
3. **Invoice posting:** cash/bank Dr (per payment mode, capped at the net amount) + Patient Receivable Dr (unpaid balance) / Income Cr (net amount). Each source is posted once (`invoice` + id + `invoice_bill_posted`). If the source changes, the old voucher is reversed and a new one posted.
4. **Refund:** Sales Returns Dr / Cash-Bank Cr, one voucher per refund receipt.
5. **Purchase paid:** Purchase expense Dr / Cash-Bank Cr. It's reversed if the payment is removed.
6. **Manual:** `/finance/income` and `/finance/expenditure` post vouchers. `/accounting/vouchers` handles manual entries (post, cancel, reverse).
7. **Financial years:** create, set active, lock.
8. **Reports** (from voucher entries only): ledger statement, cash book, bank book, day book, trial balance, profit & loss, finance summary, branch-wise revenue.

19. Printing & messaging

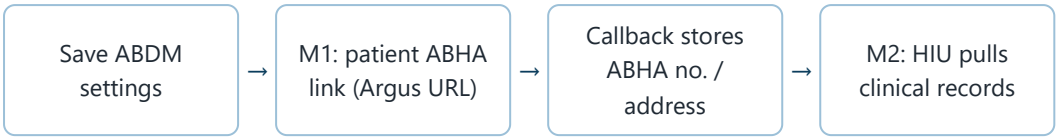
Printing (`PrintController`)

1. Settings: `GET /print/setting` and `POST /print-settings` set header, footer, margins and what to show per print type.
2. `GET /print-preview/{type}` returns HTML. `GET /print/{type}/{id}` and `/generate-print/{type}` return a PDF.
3. Types include prescription, case paper, diet chart, payment receipt, invoices, therapy package and certificates. Blade templates are in `resources/views/print` .

Messaging (`MessageService` , `MessageDispatchHelper`)

1. WhatsApp and SMS provider settings and switches for each alert: `setting/message/whatsapp` and `setting/message/sms` .
2. Messages are triggered automatically by OTP, appointment and therapy confirmations, and insurance status changes.
3. Staff can send manually: `POST setting/message/send-document` (any print) and `/send-invoice-link` .
4. Every message is logged in `/communication` and charged to the client wallet.

20. ABDM / ABHA integration (ABHA)



1. `/abdm/settings` and `/abdm/status` hold facility and connector credentials (`AbdmConnectorService`).
2. `GET /abdm/m1-url` builds the Argus M1 URL for creating or verifying a patient's ABHA. `POST /abdm/link` and `GET /abdm/patients` link and list patients.
3. Public callback `/abha/{type}` (no token) receives M1 results and serves M2 clinical data. The types are `data, historyform, prescription, invoice, generalexamination, diagnosis` , built by `AbdmClinicalRecordService` and `AbdmM2Service` .

21. Dashboards & reports (ANALYTICS)

Report	Endpoint	What it shows
Today stats	`/patient/dashboard/today-stats`	Registrations, waiting, visits and collections today
Day end	`/patient/dashboard/day-end-report`	Revenue by consulting, medicine, therapy and IPD. Therapy split OPD/IPD
Journey funnel	`/patient/dashboard/journey-funnel`	Inquiry → appointment → visit → conversion
Follow-up analytics	`/patient/analytics/followup`	Follow-ups due, done and missed
Patient master	`/patient/report`	Patient list with KPIs
Visit No. report	`/patient/visit-no-report`	Per-prescription rows, new/old, male/female, waiting time, visit revenue
Source panel / Ayush / customer	`/patient/source-panel-report`, `ayush-cover-report`, `customer-report`	Referrals, insurance cover, customer spend
Appointments / inquiries	`/appointments/report`, `/inquiries/report`	Booking and lead performance
Therapy lifecycle	`/patient/schedule-therapies/lifecycle-report`	Session progress per package
Finance	`/finance/*`, `/accounting/reports/*`	See section 18

22. Activity log & mobile app updates

Activity log

1. Important writes (invoices, medicine bills, purchase bills, certificates…) are wrapped in `runGroupedActivity`. A before/after snapshot (`ActivitySnapshotService`) is compared (`ActivityDiffService`) and saved by `ActivityAuditService`.
2. `/activity-logs` (list, filters, report) and `/activity-logs/entity` (timeline for one record) show who changed what and when.

Mobile app updates

1. Public endpoints `/mobile/app-update-config` and `/app-update-bundle` let the React Native app check for store or OTA updates before login. `/app-update-log` records the result.
2. Admins change the config per platform (`PUT /mobile/app-update-config/{platform}`) and view adoption stats. `/app-version` returns the current version.

Code map

Area	Location
Core routes	`routes/api.php`
Clinical routes	`Modules/Patient/routes/api.php`
Core services	`app/Services` (Invoice, Purchase*, Accounting/*, Inventory/*, PlanGate, UniqueIdGenerator, Message)
Clinical services	`Modules/Patient/App/Services` (PatientVisit, PatientTherapyPackage, PatientMedicineBill, Appointment, OpdPeriod, Certificate, Abdm/*)
Middleware	`app/Http/Middleware/MergeClientBranch.php`, `EnsurePlanModule.php`
Config	`config/constants.php`, `plan_modules.php`, `quotas.php`, `module_permissions.php`